

integrate-flagcx-into-nixl-backend-as-uccl-and-ucx

Proposal: Integrate FlagCX into Nixl Backend as UCCL and UCX Alternative

Date: 2026-03-12

Dev Repo: flagcx

Status: proposal

Summary

Create a `nixlFlagcxEngine` Nixl backend plugin that wraps FlagCX's heterogeneous transport layer, exposing 11+ GPU vendor support (NVIDIA, AMD, Intel, Cambricon, Ascend, Hygon, Iluvatar, MetaX, Mthreads, Enflame, etc.) to Nixl's P2P data transfer ecosystem. This would position FlagCX as a multi-vendor RDMA transport alternative to Nixl's existing single-vendor UCCL and UCX plugins, letting Nixl-based applications (particularly LLM inference serving) transparently operate across heterogeneous GPU clusters.

A dual-phase approach is proposed: Phase 0 uses FlagCX's public `flagcx.h` API (`flagcxSend` / `flagcxRecv`) for a fast PoC that validates multi-vendor GPU support with no internal FlagCX changes. Phase 1+ exposes FlagCX's net adaptor layer as a standalone library for production-grade one-sided RDMA semantics matching Nixl's transfer model.

Feasibility

API-Level Analysis

Two integration surfaces exist in FlagCX, with different trade-offs:

Surface A — Public API (`flagcx.h`): `flagcxSend` / `flagcxRecv` + `flagcxCommRegister` + `flagcxCommInitRank`

- Pros: No FlagCX internal changes needed; multi-vendor works out of the box; well-tested code path

- Cons: Two-sided semantics (both sides must actively call send/recv) vs Nixl's one-sided model (NIXL_READ/WRITE); communicator requires all ranks upfront vs Nixl's pairwise dynamic connections; no metadata export for `getPublicData()` / `loadRemoteMD()` flow; adds rendezvous latency (~1 RTT per transfer)

Surface B — Net Adaptor layer (`flagcxNetAdaptor`):

`put` / `isend` / `irecv` / `regMr` / `test`

- Pros: One-sided `put` maps directly to NIXL_WRITE; pairwise listen/connect/accept matches Nixl's model; MR handles can be serialized for metadata exchange
- Cons: Internal API, not a public contract; tightly coupled to collective pipeline initialization; requires FlagCX-side work to expose standalone

Effort & Risk

Phase	Effort	Risk	FlagCX Changes
Phase 0 (public API PoC)	small (~300 LoC)	low	none
Phase 1 (standalone net API)	large (~700-1000 LoC + FlagCX library)	medium	significant

- Prerequisites for Phase 0: A FlagCX installation with `libflagcx.so` linkable; out-of-band bootstrap for `flagcxUniqueId` distribution
- Prerequisites for Phase 1: FlagCX exposes net adaptor layer as standalone library; runtime adaptor selection; MR blob serialization protocol

Prior Art

- **Nixl UCCL plugin** (`nixl/src/plugins/uccl/uccl_backend.{h,cpp}`): The closest reference implementation. Wraps `uccl_engine_t` C API into `nixlBackendEngine`. Key patterns to reuse:
 - Listener thread for incoming connections (line 118-145 of `uccl_backend.cpp`)
 - Fifoltem-based metadata serialization for remote memory access (line 46-64 of `uccl_backend.h`)
 - Connection map indexed by `(remote_agent, gpu_index)` pair
 - `postXfer` collects scatter-gather list and calls vectorized read/write
 - Notification delivery via `uccl_engine_send_notif()` / `uccl_engine_get_notifs()`
 - Plugin boilerplate via `nixlBackendPluginCreator<nixlUcclEngine>` template

- **Nixl UCX plugin** (`nixl/src/plugins/ucx/`): The primary transport plugin. Shows multi-worker patterns and the factory design for engine variants (base, thread, thread-pool). Relevant for understanding how Nixl handles connection multiplexing and concurrent transfers.
- **Nixl libfabric plugin** (`nixl/src/plugins/libfabric/`): Demonstrates multi-rail support and explicit CUDA context management (`nixlLibfabricCudaCtx`). Relevant because FlagCX similarly needs explicit device context handling across vendors.
- **FlagCX UCX adaptor** (`flagcx/adaptor/net/ucx_adaptor.cc` , ~1300 LoC): Shows how FlagCX wraps a transport library into its `flagcxNetAdaptor` interface. Key patterns:
 - `ucxNetInit()` → IB device enumeration, UCP context/worker creation
 - `ucxNetRegMr()` → UCP memory registration, returns `flagcxNetMr_t`
 - `ucxNetIsend()/ucxNetIrecv()` → tag-matched UCP send/recv with request
 - `ucxNetTest()` → polling request completion
 - Device properties exposed via `ucxNetGetProperties()`
- **FlagCX IBRC adaptor** (`flagcx/adaptor/net/ibrc_adaptor.cc`): The default high-performance network adaptor. Uses QP-based RDMA, relevant for one-sided put operations.

Current State in FlagCX

- **Has:**
 - **Public API** (`flagcx.h`): `flagcxSend` / `flagcxRecv` (P2P), `flagcxCommInitRank` (communicator), `flagcxCommRegister` / `flagcxCommDeregister` (buffer registration), `flagcxCommWindowRegister` (one-sided windows), `flagcxGroupStart` / `flagcxGroupEnd` (operation batching), `flagcxMemAlloc` / `flagcxMemFree` (symmetric memory)
 - Well-defined `flagcxNetAdaptor` C struct with function pointers (init, listen, connect, accept, regMr, deregMr, isend, irecv, test, close, put, putSignal, waitValue, iflush) — defined in `flagcx/include/flagcx_net.h`
 - Multiple network adaptor implementations: IBRC (default), Socket, UCX, IBUC — all sharing the same interface
 - Device adaptor abstraction (`flagcxDeviceAdaptor`) for multi-vendor GPU memory management (malloc, memcpy, streamCreate, etc.)
 - Proxy thread infrastructure for asynchronous network operations (three-stage send pipeline, four-stage recv pipeline)
 - IPC-based P2P transport for intra-node communication
 - Existing knowledge of UCCL (referenced in `ib_retrans.cc` , `ib_common.h`)

- **Missing** (for Phase 1+ only — Phase 0 doesn't need these):
 - Standalone library packaging of the net adaptor layer — currently net adaptors are initialized as part of the collective pipeline (`flagcxTransportP2pSetup`)
 - A C API for standalone net adaptor usage (similar to UCCL's `ucccl_engine_t` C API)
 - Metadata serialization primitives for exporting memory registration handles across process boundaries (Nixl requires `getPublicData()` to produce a serializable blob)
 - Connection info export/import mechanism (Nixl's `getConnInfo()` / `loadRemoteConnInfo()` pattern)
 - Transfer request handle abstraction for tracking individual P2P operations (FlagCX tracks operations inside proxy thread state, not as user-visible handles)
- **Key files that would be modified/created:**
 - Phase 0: `nixl/src/plugins/flagcx/flagcx_backend.{h,cpp,plugin.cpp}` — Nixl backend plugin using public API only
 - Phase 1+: `flagcx/include/flagcx_net_standalone.h` — new public C API for standalone net adaptor usage
 - Phase 1+: `flagcx/core/net_standalone.cc` — implementation: init adaptor, register mem, export metadata, P2P transfers
 - Phase 1+: `flagcx/adaptor/net/ibrc_adaptor.cc` (+ others) — minor changes to support standalone initialization

Proposed Approach

Phase 0: Public API PoC — Two-sided with Transfer Agent (Nixl-side only)

No FlagCX internal changes. Uses `flagcx.h` public API exclusively.

Architecture:

代码块

```

1  nixlFlagcxEngine (per Nixl agent)
2  |— flagcxComm_t           ← world communicator, all agents
3  |— transfer_agent thread  ← posts matching recv/send for remote-
    initiated ops
4  |— control_comm (flagcxComm_t) ← separate comm for control messages
5  |— registered_bufs map     ← (addr → flagcx handle) via
    flagcxCommRegister
6  |— pending_reqs queue      ← outstanding transfer requests
  
```

Nixl SB API mapping (Phase 0):

Nixl Method	FlagCX Public API	Notes
Constructor	<code>flagcxGetUniqueId</code> + <code>flagcxCommInitRank</code> × 2 (data + control)	Requires bootstrap: all agents exchange UniqueId before engine init
<code>supportsRemote()</code>	<code>true</code>	
<code>supportsLocal()</code>	<code>false</code>	
<code>supportsNotif()</code>	<code>true</code>	Via control comm messages
<code>getSupportedMems()</code>	<code>{DRAM_SEG, VRAM_SEG}</code>	Multi-vendor via FlagCX device adaptor
<code>registerMem()</code>	<code>flagcxCommRegister(comm, buf, size, &handle)</code>	Store (addr, size, handle) in local map
<code>deregisterMem()</code>	<code>flagcxCommDeregister(comm, handle)</code>	
<code>getPublicData()</code>	Serialize (rank, buf_addr, size) tuple	No MR blob — rank-based addressing replaces RDMA keys
<code>getConnInfo()</code>	Serialize (rank, hostname, gpu_id)	Comm already connected; this is informational
<code>loadRemoteConnInfo()</code>	No-op (comm handles connections)	Connection implicit in <code>flagcxCommInitRank</code>
<code>loadRemoteMD()</code>	Deserialize (remote_rank, buf_addr, size)	Store for transfer agent coordination
<code>connect()</code>	No-op (comm already connected)	
<code>prepXfer()</code>	Build transfer request: (op, local_addr, remote_rank, remote_addr, size)	
<code>postXfer(NIXL_WRITE)</code>	Send control msg to remote → remote agent posts <code>flagcxRecv</code> → local calls <code>flagcxSend</code>	~1 RTT rendezvous overhead
<code>postXfer(NIXL_READ)</code>		~1 RTT rendezvous overhead

	Send control msg to remote → remote agent posts <code>flagcxSend</code> → local calls <code>flagcxRecv</code>	
<code>checkXfer()</code>	<code>flagcxStreamQuery(stream)</code> or event-based polling	Stream-based completion
<code>getNotifs()</code>	Receive control messages from control comm	
<code>genNotif()</code>	Send control message via control comm	

Transfer agent thread (runs on every rank):

代码块

```

1  while running:
2      poll control_comm for incoming requests
3      for each request:
4          if request.op == REMOTE_WRITE_TO_ME:
5              flagcxRecv(my_buf + request.offset, request.count,
6                          request.dtype, request.src_rank, data_comm, stream)
7          elif request.op == REMOTE_READ_FROM_ME:
8              flagcxSend(my_buf + request.offset, request.count,
9                          request.dtype, request.dst_rank, data_comm, stream)

```

Key design decisions for Phase 0:

- **Two communicators:** Data comm for bulk transfers, control comm for coordination messages. Avoids head-of-line blocking between control and data.
- **Bootstrap via etcd/env:** Nixl agents already exchange metadata via etcd or file. FlagCX Uniqueld can be distributed through the same channel. Store rank assignment in Nixl agent metadata.
- **Group semantics for batching:** Multiple concurrent transfers within `flagcxGroupStart` / `flagcxGroupEnd` to amortize rendezvous cost.
- **Stream-based completion:** FlagCX enqueues on streams; use `flagcxStreamQuery` or events for non-blocking completion check.

Limitations of Phase 0:

- ~1 RTT rendezvous latency per transfer (control msg + matching send/recv posting)
- All agents must participate in `flagcxCommInitRank` at startup (no dynamic join)

- No true one-sided RDMA — every transfer requires active participation from both sides
- `flagcxCommRegister` handle is opaque — metadata exchange carries rank+addr, not MR keys

Phase 0 validates: Multi-vendor GPU memory transfers work end-to-end through Nixl's API. This is the primary value proposition — if AMD/Intel/Cambricon GPUs can transfer data via Nixl using FlagCX, the integration has clear value regardless of latency.

Phase 1: FlagCX Standalone Net API (FlagCX-side)

After Phase 0 validates the value, extract FlagCX's net adaptor for production-grade one-sided RDMA.

1. Design `flagcx_net_standalone.h` C API modeled after UCCL's `uccl_engine.h`:

代码块

```
1  flagcxResult_t flagcxNetStandaloneInit(flagcxNetHandle_t *handle, int
    deviceIndex);
2  flagcxResult_t flagcxNetStandaloneListen(flagcxNetHandle_t handle, char
    *connInfo, size_t *connInfoSize);
3  flagcxResult_t flagcxNetStandaloneConnect(flagcxNetHandle_t handle, const char
    *remoteConnInfo, flagcxNetConn_t *conn);
4  flagcxResult_t flagcxNetStandaloneAccept(flagcxNetHandle_t handle,
    flagcxNetConn_t *conn);
5  flagcxResult_t flagcxNetStandaloneRegMem(flagcxNetHandle_t handle, void *ptr,
    size_t size, flagcxNetMr_t *mr);
6  flagcxResult_t flagcxNetStandaloneDeregMem(flagcxNetHandle_t handle,
    flagcxNetMr_t mr);
7  flagcxResult_t flagcxNetStandaloneGetMrBlob(flagcxNetMr_t mr, void *blob,
    size_t *blobSize);
8  flagcxResult_t flagcxNetStandaloneLoadMrBlob(flagcxNetHandle_t handle, const
    void *blob, size_t blobSize, flagcxNetMr_t *mr);
9  flagcxResult_t flagcxNetStandalonePut(flagcxNetConn_t conn, flagcxNetMr_t
    localMr, size_t localOff, flagcxNetMr_t remoteMr, size_t remoteOff, size_t
    size, flagcxNetReq_t *req);
10 flagcxResult_t flagcxNetStandaloneGet(flagcxNetConn_t conn, flagcxNetMr_t
    localMr, size_t localOff, flagcxNetMr_t remoteMr, size_t remoteOff, size_t
    size, flagcxNetReq_t *req);
11 flagcxResult_t flagcxNetStandaloneTest(flagcxNetReq_t req, int *done);
12 flagcxResult_t flagcxNetStandaloneFinalize(flagcxNetHandle_t handle);
```

2. **Implement standalone initialization** that selects and initializes the appropriate net adaptor based on runtime detection (not compile-time `#ifdef`), using the existing adaptor registry. This is the key architectural change — it decouples net adaptor usage from the collective pipeline.
3. **Add memory registration metadata export/import** (`GetMrBlob` / `LoadMrBlob`): serialize the adaptor-specific MR handle (e.g., rkey for IBRC, UCP memh for UCX) into a portable blob that can cross Nixl's agent metadata exchange boundary.

Phase 2: Production Nixl Backend Plugin (Nixl-side)

4. **Upgrade `nixlFlagcxEngine`** to use standalone net API instead of public API:
- Replace `flagcxComm` with pairwise `flagcxNetStandaloneConnect`
 - Replace transfer agent thread with direct one-sided put/get
 - Replace rank+addr metadata with actual MR blobs
 - Remove rendezvous overhead — true one-sided RDMA
 - Keep Phase 0 code as fallback for adaptors that don't support `put` (e.g., Socket)
5. **Plugin entry** using `nixlBackendPluginCreator<nixlFlagcxEngine>` with name "FlagCX", supporting DRAM_SEG and VRAM_SEG.

Phase 3: Integration & Testing

6. **Build system integration:** Link the Nixl FlagCX plugin against `libflagcx.so` (Phase 0) or `libflagcx_net_standalone.so` (Phase 1+). Make it a conditional build target (like GDS, GPUNetIO).
7. **Test with heterogeneous GPUs:** Validate on a cluster with mixed GPU vendors — the primary value proposition. Start with single-vendor validation (NVIDIA-to-NVIDIA via IBRC), then test cross-vendor.

Phase Comparison

Aspect	Phase 0 (Public API)	Phase 1+ (Net Adaptor)
FlagCX changes	None	New standalone library
Transfer semantics	Two-sided (send/recv)	One-sided (put/get)
Latency overhead	~1 RTT rendezvous	Near-zero (direct RDMA)
Connection model	All-at-once <code>CommInitRank</code>	Pairwise listen/connect
Metadata exchange	rank + addr tuples	Serialized MR blobs

Dynamic join	No	Yes
Best for	Bulk transfers, validation	Latency-sensitive inference
Implementation time	Days	Weeks

Alternatives Considered

A. Skip Phase 0 — go directly to standalone net API.

Not recommended: Phase 0 costs days, not weeks, and proves the value proposition (multi-vendor GPU transfers through Nixl) before investing in FlagCX internal changes. If Phase 0 reveals the integration isn't useful, Phase 1 effort is saved entirely.

B. Use only the public API permanently (skip net adaptor extraction).

Viable for bulk transfer workloads (checkpointing, model loading) where ~1 RTT rendezvous latency is acceptable. Not viable for Nixl's primary use case (inference KV cache transfer) where microsecond latency matters. Keep Phase 0 as a fallback path for adaptors without one-sided support.

C. Implement FlagCX as a UCX transport (UCT layer) instead of a Nixl plugin.

Rejected: This would make FlagCX a UCX dependency rather than a Nixl peer. The value proposition is offering an alternative to UCX, not being consumed by it. Also, UCT's vtable-based dispatch is too rigid for FlagCX's adaptor selection model.

D. Use FlagCX's proxy thread for async operations instead of direct adaptor calls.

Considered for Phase 1: The proxy thread adds latency (queuing + wakeup) but provides better CPU utilization under load. Phase 1 should initially use direct adaptor calls (lower latency for inference workloads), with proxy thread as a future optimization for high-throughput training workloads.

E. Build a bidirectional integration (Nixl as FlagCX backend + FlagCX as Nixl backend).

Future work: This proposal focuses on FlagCX → Nixl direction. The reverse (Nixl as a FlagCX net adaptor) is a natural follow-on but would require FlagCX to support dlopen-based adaptor loading, which aligns with the existing `runtime-adaptor-hot-swap` insight.

First Step

Build Phase 0 — the public API PoC. Minimal viable `nixlFlagcxEngine` using `flagcx.h`:

- Skeleton plugin** (~150 LoC): `flagcx_backend.h` + `flagcx_backend.cpp` + `flagcx_plugin.cpp` implementing `nixlBackendEngine` with stubbed methods, linking against `libflagcx.so`
- Bootstrap integration**: Use Nixl's existing metadata exchange (etcd or file) to distribute `flagcxUniqueId` and assign ranks. Each `nixlFlagcxEngine` instance calls `flagcxCommInitRank` during construction.

3. **Register + transfer:** Implement `registerMem` → `flagcxCommRegister`, `postXfer` → `flagcxSend` / `flagcxRecv` with a transfer agent thread
4. **Two-process test:** Agent A registers VRAM buffer on GPU vendor X → Agent B reads it via `NIXL_READ` → verify correctness

Success criteria: correct data transfer between two Nixl agents using FlagCX as the backend, on at least one GPU vendor. Latency measurement to quantify the rendezvous overhead vs. UCCL baseline.

If Phase 0 validates, proceed to Phase 1 (standalone net API) for production latency.

Risks & Mitigations

Risk	Phase	Likelihood	Mitigation
<code>flagcxCommInitRank</code> requires all agents at startup — can't dynamically add agents	0	High	Accept this limitation for PoC; document that all Nixl agents using FlagCX backend must be known at init time. Phase 1 eliminates this with pairwise connect.
Rendezvous overhead makes Phase 0 unacceptable for inference latency	0	High	Expected and accepted — Phase 0 is for validation, not production. Use <code>flagcxGroupStart/End</code> to batch multiple transfers and amortize overhead.
Control channel (second <code>flagcxComm</code>) adds resource overhead	0	Medium	Control messages are tiny; a lightweight socket-based control channel could replace the second comm if resource pressure is an issue.
<code>flagcxCommRegister</code> doesn't work for all memory types across vendors	0	Low	FlagCX's device adaptor handles vendor-specific registration internally; if a vendor doesn't support registration,

			fall back to staged copy through host memory.
FlagCX net adaptor is too tightly coupled to collective pipeline	1	Medium	Phase 0 validates value first; if Phase 1 decoupling is blocked, keep Phase 0 as permanent solution for bulk transfers.
Compile-time vendor selection prevents runtime multi-vendor support	1	High	Phase 1 step 2 specifically addresses this; worst case, build separate .so per vendor and let Nixl's plugin manager load the right one.
MR handle serialization is adaptor-specific and fragile	1	Medium	Define a tagged blob format (adaptor_type + version + payload); UCCL plugin's hex-encoded fifo_item approach works as reference.
Performance regression vs. native UCCL/UCX due to additional abstraction layer	1	Medium	Benchmark Phase 1 against raw IBRC; the indirection cost should be negligible (one function pointer dispatch).
Maintenance burden: changes to FlagCX internal adaptor interface break the standalone API	1	High	Define the standalone API as a stable C API with versioning; add CI tests; keep the API surface minimal.
Multi-vendor GPU interop: e.g., AMD GPU memory may not be RDMA-accessible by NVIDIA NIC	0,1	Medium	Hardware limitation, not software; document supported topology matrices; start with homogeneous validation per vendor.

Related

- Knowledge: `nixl/architecture.md`, `nixl/api.md`, `nixl/transport.md`, `nixl/memory.md`, `nixl/plugins.md`, `UCCL/architecture.md`, `UCCL/transport.md`, `UCX/architecture.md`, `UCX/transport.md`, `FlagCX/architecture.md`, `FlagCX/transport.md`, `FlagCX/api.md`
- Analysis: transport-related analysis files in `~/WORKDIR/.ccl/analysis/`
- Existing insights: `dual-mode-transport-adaptor` (P1 — related: also about FlagCX transport extensibility), `runtime-adaptor-hot-swap` (P1 innovation — prerequisite: FlagCX needs runtime adaptor selection for this proposal to reach full potential)
- Source code references: `nixl/src/plugins/uccl/uccl_backend.{h,cpp}` (primary reference implementation), `flagcx/adaptor/net/ucx_adaptor.cc` (FlagCX's existing UCX integration pattern), `flagcx/adaptor/net/ibrc_adaptor.cc` (target adaptor for PoC)